Category C Problems

Category C · 5 problems selected for logic-building using arrays, loops, and conditionals only

L1. Two Sum

Scenario

A shopkeeper wants to find two items from a list of prices that together add up to exactly a customer's budget. With only a handful of items on the shelf, the simplest approach — checking every possible pair — is more than fast enough.

Task

- Accept an integer array, nums, and an integer target.
- Using two nested loops, check every pair of different positions (i, j) in the array.
- If nums[i] + nums[j] equals target, return the two indices [i, j] immediately.
- You may assume the input always has exactly one valid pair, and you cannot use the same element twice.

Suggested Method Signature(s)

```
int[] twoSum(int[] nums, int target)
```

Sample Input / Output

Input	Output
nums = [2, 7, 11, 15], target = 9	[0, 1] (nums[0] + nums[1] = 2 + 7 = 9)
nums = [3, 2, 4], target = 6	[1, 2] (nums[1] + nums[2] = 2 + 4 = 6)

Concepts covered: *Nested for loops, array indexing, basic pairwise search logic.*

L2. Best Time to Buy and Sell Stock

Scenario

A trainee investor has one week of daily stock prices and wants to know the single best day to buy and the single best later day to sell, to make the largest possible profit — found in one simple pass through the prices, left to right.

Task

- Accept an integer array prices, where prices[i] is the price on day i.
- Walk through the array once, keeping track of the lowest price seen so far.
- At each day, work out the profit you'd make if you sold today: today's price minus the lowest price seen so far.
- Keep a running record of the largest such profit seen across the whole array.
- If the price only ever goes down, the answer is 0 — there's no profitable day to sell.

Suggested Method Signature(s)

```
int maxProfit(int[] prices)
```

Sample Input / Output

Input	Output
prices = [7, 1, 5, 3, 6, 4]	5 (buy on day 2 at price 1, sell on day 5 at price 6)
prices = [7, 6, 4, 3, 1]	0 (prices only fall, so no trade is profitable)

Concepts covered: *Single-pass array traversal, tracking a running minimum and a running maximum together, basic comparison logic.*

L3. Contains Duplicate

Scenario

Before finalizing an exam seating chart, the office must double-check that no roll number was accidentally entered twice in the list — by comparing every entry against every other entry.

Task

- Accept an integer array nums.
- Using two nested loops, compare every element at position i against every element at a different position j.
- If any two different positions hold the exact same value, return true right away.
- If no matching pair is found after checking every possible pair, return false.

Suggested Method Signature(s)

```
boolean containsDuplicate(int[] nums)
```

Sample Input / Output

Input	Output
nums = [1, 2, 3, 1]	true (the value 1 appears at two different positions)
nums = [1, 2, 3, 4]	false (every value is distinct)

Concepts covered: *Nested loops, pairwise comparison, boolean return logic, early exit from a loop once an answer is found.*

L4. Merge Two Sorted Arrays

Scenario

Two class sections each submit their exam scores already sorted from lowest to highest. The examination office needs one single combined sorted list — without throwing both lists together and re-sorting everything from scratch.

Task

- Accept two sorted integer arrays, arr1 and arr2.
- Keep one index pointer for each array, both starting at 0, and create a new empty result array.
- Using a while loop, repeatedly compare the current elements pointed to in arr1 and arr2, copy the smaller one into the result array, and move that array's pointer forward by one.
- Once one array is fully copied over, copy all of the remaining elements from the other array directly onto the end of the result.
- Return the fully merged, fully sorted result array.

Suggested Method Signature(s)

```
int[] mergeSortedArrays(int[] arr1, int[] arr2)
```

Sample Input / Output

Input	Output
arr1 = [1, 3, 5], arr2 = [2, 4, 6]	[1, 2, 3, 4, 5, 6]
arr1 = [], arr2 = [1, 2, 3]	[1, 2, 3]

Concepts covered: *Two-index (two-pointer) array traversal, while loops, comparing and copying elements between arrays, handling a fully-consumed array.*

L5. Rotate Array

Scenario

A playlist needs to be shifted so the last few songs move to the front of the queue, rotating the whole list to the right by a given number of positions — solved the most direct way: work out exactly where every song lands, and build the new order from scratch.

Task

- Accept an integer array nums and an integer k, the number of positions to rotate to the right.
- First reduce k using k = k % nums.length — rotating by the array's own length (or a multiple of it) should have no visible effect at all.
- Create a new array of the same size. For every index i in the original array, work out its new position after rotation, and place nums[i] there: newArray[(i + k) % nums.length] = nums[i].
- Copy the values from the new array back into nums (or simply return the new array, whichever your setup expects).

Suggested Method Signature(s)

```
int[] rotateArray(int[] nums, int k)
```

Sample Input / Output

Input	Output
nums = [1, 2, 3, 4, 5, 6, 7], k = 3	[5, 6, 7, 1, 2, 3, 4]
nums = [1, 2], k = 3	[2, 1] (k % length = 3 % 2 = 1, a single rotation)

Concepts covered: *Modulo arithmetic for wraparound indexing, building a new array from calculated positions, basic array copying.*